

SHAP & Interpretabilidade

Guia de Estudo para Modelagem de Risco de Crédito

Hamilton Nascimento

Projeto de Portfólio — Ciência de Dados aplicada a Crédito · 2026

Conceitos · Visualizações · Implementação em Python

Índice

- 1 Por que interpretabilidade importa em crédito**
O problema do modelo caixa-preta e suas consequências reais
 - 2 Fundamentos do SHAP**
O que são SHAP values, de onde vêm e o que medem
 - 3 Tipos de Explainers**
TreeExplainer, LinearExplainer e quando usar cada um
 - 4 Visualizações SHAP**
Summary plot, waterfall, force plot, dependence plot e beeswarm
 - 5 Importância global vs local**
Analisar o modelo como um todo e explicar decisões individuais
 - 6 SHAP aplicado a risco de crédito**
Conectando os valores SHAP à linguagem de negócio
 - 7 Simulação de política de concessão**
Threshold, trade-offs e cenários conservador/moderado/agressivo
 - 8 Implementação passo a passo em Python**
Código completo e comentado para o projeto Give Me Some Credit
 - 9 Erros comuns e boas práticas**
O que evitar e o que demonstra maturidade analítica
-

1

Por que interpretabilidade importa em crédito

O problema do modelo caixa-preta

Modelos de machine learning modernos — como LightGBM, XGBoost e redes neurais — são capazes de capturar padrões complexos nos dados e atingir métricas excelentes. Mas eles têm um custo: são caixas-pretas. Recebem dados, produzem uma previsão, e não explicam o raciocínio por trás dela.

Em domínios como crédito, isso cria problemas reais que vão além da performance do modelo.

Problemas práticos do modelo opaco em crédito

Regulação	Bancos e fintechs são obrigados por lei a explicar recusas de crédito. No Brasil, o Banco Central exige que o cliente possa questionar decisões automatizadas. Um modelo que não explica suas decisões cria risco regulatório.
Confiança do negócio	Um gerente de risco não vai homologar um modelo que ele não consegue entender. A interpretabilidade é pré-requisito para que o modelo saia do notebook e vá para produção.
Detecção de viés	Modelos opacos podem aprender correlações espúrias — por exemplo, usar CEP como proxy de raça. Sem interpretabilidade, esse viés é invisível.
Debugging	Quando o modelo começa a errar mais do que o esperado, é preciso entender por quê. Sem explicações, o diagnóstico é impossível.

Conceito-chave

Interpretabilidade não é opcional em crédito — é parte do produto. Um modelo com AUC 0.85 que ninguém entende tem menos valor do que um modelo com AUC 0.80 que o time de negócio consegue usar com confiança.

Interpretabilidade global vs local

Existem dois tipos de pergunta que queremos responder sobre um modelo:

- **Global:** quais variáveis o modelo considera mais importantes no geral? Como cada variável influencia as previsões em média? Isso é a visão macro do comportamento do modelo.
- **Local:** por que o modelo deu este score específico para este cliente? Quais variáveis pesaram mais nesta decisão individual? Isso é o que permite explicar uma recusa para o cliente ou para o regulador.

O SHAP responde as duas perguntas com o mesmo framework — essa é sua principal vantagem sobre outras técnicas.

2

Fundamentos do SHAP

O que são SHAP values, de onde vêm e o que medem

O que é SHAP

SHAP (SHapley Additive exPlanations) é um método para explicar a saída de qualquer modelo de machine learning. Ele foi desenvolvido por Lundberg e Lee (2017) com base na teoria dos jogos cooperativos — especificamente nos Shapley values de Lloyd Shapley (Nobel de Economia, 2012).

A ideia intuitiva: divisão justa de crédito

Imagine um time de jogadores que ganhou um prêmio juntos. Como dividir o prêmio de forma justa entre os jogadores, considerando a contribuição de cada um? Essa é exatamente a pergunta que os Shapley values respondem — e o SHAP adapta essa lógica para modelos de machine learning.

No contexto do modelo, os 'jogadores' são as features. O 'prêmio' é a previsão do modelo para um cliente específico. O SHAP calcula quanto cada feature contribuiu para que a previsão fosse maior ou menor do que a média.

Definição central

SHAP value de uma feature = quanto essa feature empurrou a previsão para cima (risco maior) ou para baixo (risco menor) em relação à previsão média do modelo.

A equação fundamental

Para qualquer cliente, o SHAP garante que:

$$\text{Previsão do cliente} = \text{Previsão média do modelo} + \text{Soma dos SHAP values de todas as features}$$

Exemplo concreto — cliente de risco de crédito

Suponha que a previsão média do modelo para toda a base seja 0.067 (6,7% de chance de inadimplência). Para um cliente específico, o modelo prevê 0.42 (42% de chance). Os SHAP values explicam essa diferença de 0.353:

Feature	Valor do cliente	SHAP value	Interpretação
RevolvingUtilization	92%	+0.180	Empurrou muito o risco para cima
NumberOfTime30-59	4 atrasos	+0.120	Histórico ruim — risco maior
MonthlyIncome	R\$ 1.800	+0.045	Renda baixa — risco levemente maior
age	28 anos	+0.022	Cliente jovem — risco levemente maior
NumberOfDependents	1	-0.014	Poucos dependentes — reduz risco
DebtRatio	0.35	+0.000	Neutro — não influenciou

TOTAL		+0.353	$0.067 + 0.353 = 0.420$ ✓
-------	--	--------	---------------------------

■ *SHAP values positivos aumentam o score (mais risco). SHAP values negativos diminuem o score (menos risco). A soma sempre fecha com a previsão do modelo.*

Propriedades que tornam o SHAP confiável

- **Eficiência:** A soma dos SHAP values sempre iguala a diferença entre a previsão do cliente e a previsão média. Sem resíduo inexplicado.
- **Simetria:** Features com contribuições idênticas recebem SHAP values idênticos. Não há favoritismo arbitrário.
- **Dummy:** Uma feature que não afeta a previsão recebe SHAP value zero.
- **Aditividade:** O SHAP value de uma feature em um ensemble é a média ponderada dos SHAP values nos modelos individuais.

3

Tipos de Explainers

TreeExplainer, LinearExplainer e quando usar cada um

A biblioteca SHAP oferece diferentes explainers otimizados para cada tipo de modelo. Escolher o explainer certo é importante para eficiência e precisão.

TreeExplainer

Modelos: LightGBM, XGBoost, Random Forest, Decision Tree

Precisão: ★★★ Exato |

Velocidade: Rápido

Algoritmo exato e eficiente para modelos baseados em árvore. É o que você vai usar neste projeto com o LightGBM.

LinearExplainer

Modelos: Regressão Logística, Ridge, Lasso

Precisão: ★★★ Exato |

Velocidade: Muito rápido

Exploita a estrutura linear do modelo. Use para explicar o baseline de regressão logística.

KernelExplainer

Modelos: Qualquer modelo

Precisão: ★★ ■ Aproximado |

Velocidade: Lento

Funciona com qualquer modelo mas é muito lento. Evitar em bases grandes — usar apenas quando não há alternativa.

DeepExplainer

Modelos: Redes neurais (TensorFlow, PyTorch)

Precisão: ★★ ■ Aproximado |

Velocidade: Médio

Específico para deep learning. Não aplicável neste projeto.

Como instanciar cada explainer

```
import shap

# TreeExplainer – para LightGBM (modelo principal)
explainer = shap.TreeExplainer(lgbm_model)
shap_values = explainer.shap_values(X_test)

# Para modelos binários, shap_values é uma lista de 2 arrays
# shap_values[0] = contribuições para classe 0 (adimplente)
# shap_values[1] = contribuições para classe 1 (inadimplente)
# Sempre usar shap_values[1] em problemas de risco de crédito

# LinearExplainer – para Regressão Logística (baseline)
explainer_lr = shap.LinearExplainer(logistic_model, X_train)
shap_values_lr = explainer_lr.shap_values(X_test)
```

■ Calcular `shap_values` pode ser lento para bases grandes. Para visualizações, use uma amostra de 500 a 2.000 observações: `X_sample = X_test.sample(1000, random_state=42)`

4

Visualizações SHAP

Summary plot, waterfall, beeswarm e dependence plot

O SHAP oferece um conjunto rico de visualizações. Cada uma responde uma pergunta diferente. Neste projeto, você vai usar quatro delas — descritas em detalhe abaixo.

1. Beeswarm Plot (Summary Plot) — visão global

É a visualização mais usada em análises de importância global. Mostra todas as features ordenadas por importância média e, para cada feature, a distribuição dos SHAP values de todos os clientes.

Como ler:

- Cada ponto é um cliente
- Posição horizontal = SHAP value (direita = mais risco, esquerda = menos risco)
- Cor = valor original da feature (vermelho = alto, azul = baixo)
- Features mais importantes ficam no topo

```
# Beeswarm plot
shap.summary_plot(
    shap_values[1], # classe 1 = inadimplente
    X_sample,
    plot_type='dot', # 'dot' = beeswarm (padrão recomendado)
    max_display=10, # mostrar top 10 features
    show=False
)
plt.title('Importância Global das Features – SHAP Beeswarm')
plt.tight_layout()
plt.savefig('reports/figures/shap_beeswarm.png', dpi=150, bbox_inches='tight')
plt.close()
```

Como narrar o gráfico

Exemplo de insight: 'RevolvingUtilization tem pontos vermelhos (alta utilização) fortemente deslocados para a direita — confirma que alta utilização do rotativo aumenta consistentemente o risco de inadimplência.'

2. Waterfall Plot — explicação individual

Explica a previsão de um único cliente. Mostra como cada feature empurrou o score para cima ou para baixo a partir da previsão média do modelo, chegando até a previsão final.

Como ler:

- Começa na previsão média ($E[f(x)]$) na parte inferior
- Barras vermelhas empurram para cima (aumentam o risco)
- Barras azuis empurram para baixo (reduzem o risco)

- Termina na previsão final do cliente ($f(x)$) no topo

```
# Waterfall plot para um cliente específico
# Primeiro, criar o objeto Explanation (API moderna do SHAP)
explainer = shap.TreeExplainer(lgbm_model)
explanation = explainer(X_sample) # nota: sem .shap_values()

# Escolher índice do cliente (ex: cliente de alto risco)
idx_alto_risco = y_prob_sample.argmax() # maior score
idx_baixo_risco = y_prob_sample.argmin() # menor score

# Plotar
shap.plots.waterfall(explanation[idx_alto_risco], show=False)
plt.title('Explicação Individual – Cliente de Alto Risco')
plt.tight_layout()
plt.savefig('reports/figures/shap_waterfall_alto_risco.png',
            dpi=150, bbox_inches='tight')
plt.close()
```

3. Bar Plot — importância média absoluta

Versão simplificada do beeswarm — mostra apenas a importância média de cada feature (média dos |SHAP values| absolutos). Mais fácil de ler para apresentações executivas.

```
shap.summary_plot(
    shap_values[1],
    X_sample,
    plot_type='bar', # barras de importância média
    max_display=10,
    show=False
)
plt.title('Importância Média das Features (|SHAP|)')
plt.savefig('reports/figures/shap_bar.png', dpi=150, bbox_inches='tight')
plt.close()
```

4. Dependence Plot — relação entre feature e SHAP value

Mostra como o SHAP value de uma feature varia em função do seu valor real. Útil para identificar threshold naturais e relações não-lineares.

```
# Dependence plot para utilização do rotativo
shap.dependence_plot(
    'RevolvingUtilizationOfUnsecuredLines',
    shap_values[1],
    X_sample,
    interaction_index=None, # sem coloração por interação
```



```
show=False
)
plt.title('SHAP vs Utilização do Rotativo')
plt.savefig('reports/figures/shap_dependence_util.png',
            dpi=150, bbox_inches='tight')
plt.close()
```

Quando usar

Use o dependence plot nas variáveis mais importantes identificadas no beeswarm. Ele frequentemente revela o threshold natural da variável — por exemplo, o ponto exato onde a utilização do rotativo começa a aumentar o risco.

5

Importância Global vs Local

Analisar o modelo como um todo e explicar decisões individuais

Dimensão	Análise Global	Análise Local
Pergunta	Quais features o modelo usa mais?	Por que esse cliente específico recebeu esse score?
Visualização principal	Beeswarm plot, Bar plot	Waterfall plot, Force plot
Audiência	Time de modelagem, gestores de risco	Analista de crédito, regulador, cliente
Código SHAP	<code>shap.summary_plot(...)</code>	<code>shap.plots.waterfall(explanation[i])</code>
Uso no projeto	Sprint 5 — primeiro passo	Sprint 5 — segundo passo

Estratégia para o projeto

A ordem correta de análise é sempre global primeiro, depois local:

Passo 1	Beeswarm global Identificar quais features o modelo considera mais importantes. Verificar se o ranking faz sentido do ponto de vista de negócio — se uma feature irrelevante aparece no topo, pode ser sinal de data leakage.
Passo 2	Narrativa global Para as top 5 features, escrever uma frase de insight conectando o SHAP value com a lógica de crédito.
Passo 3	Selecionar clientes para análise local Escolher 1 cliente de alto risco (recusado) e 1 de baixo risco (aprovado). Usar <code>y_prob</code> para encontrá-los.
Passo 4	Waterfall individual Para cada cliente, plotar o waterfall e escrever a explicação em linguagem de negócio — como se estivesse explicando para o próprio cliente.

6

SHAP aplicado a risco de crédito

Conectando SHAP values à linguagem de negócio

O maior valor do SHAP não está nos gráficos — está na narrativa. Saber calcular e plotar é necessário, mas insuficiente. O diferencial está em traduzir o que os valores significam em decisões de negócio.

Como traduzir SHAP values em linguagem de crédito

SHAP value alto e positivo em Utilização

→ O cliente está usando X% do limite disponível. Isso sugere dependência do crédito rotativo — clientes nessa situação têm Y% mais chance de inadimplência.

SHAP value alto e positivo em Histórico de atrasos

→ O cliente registrou N atrasos de 30-59 dias no período observado. Comportamento histórico de pagamento é o preditor mais direto de risco futuro — esse fator foi o principal responsável pela recusa.

SHAP value negativo em Renda

→ A renda mensal do cliente (R\$ X) é superior à mediana da base. Esse fator contribuiu para reduzir o score de risco — clientes com renda mais alta tendem a honrar compromissos.

SHAP value próximo de zero

→ Esta variável não influenciou materialmente a decisão para este cliente. Seu valor está próximo da mediana da base — sem sinal diferenciador.

Template de explicação individual

Use este template para documentar cada cliente analisado no projeto:

```
# Cliente [ID] — Score: X.XX (Decisão: RECUSADO / APROVADO)
#
# Previsão média do modelo: 0.067 (6.7%)
# Previsão deste cliente: 0.420 (42.0%)
# Diferença explicada: +0.353
#
# Principais fatores que AUMENTARAM o risco:
# 1. RevolvingUtilization = 92% → SHAP: +0.180
# Utilização muito alta sinaliza estresse financeiro
#
# 2. NumberOfTime30-59 = 4 → SHAP: +0.120
# Histórico de atrasos recorrentes — principal fator de risco
#
# Principais fatores que REDUZIRAM o risco:
# 1. NumberOfDependents = 1 → SHAP: -0.014
# Poucos dependentes reduzem levemente a pressão financeira
```

#

Narrativa executiva:

Este cliente foi recusado principalmente pelo histórico de

atrasos recorrentes (4 ocorrências) combinado com utilização

muito alta do crédito rotativo (92%). Esses dois fatores juntos

respondem por 85% do aumento de risco em relação à média.

7

Simulação de Política de Concessão

Threshold, trade-offs e cenários de aprovação

Após entender o modelo via SHAP, o próximo passo é traduzir os scores em uma política de crédito operacional. Isso envolve escolher um threshold — um ponto de corte no score a partir do qual o cliente é recusado.

O que é o threshold e por que importa

O modelo produz uma probabilidade contínua entre 0 e 1 para cada cliente. Para tomar uma decisão binária (aprovar ou recusar), é preciso definir um corte:

```
threshold = 0.30 # exemplo

y_pred = (y_prob >= threshold).astype(int)

# 1 = recusado (score acima do threshold = alto risco)
# 0 = aprovado (score abaixo do threshold = baixo risco)
```

Threshold baixo (ex: 0.10) → mais recusas → carteira mais segura, menor volume. Threshold alto (ex: 0.50) → menos recusas → carteira maior, mais risco.

Os três cenários do projeto

Cenário	Threshold	Volume aprovado	Taxa inadimpl.	Segurança	Quando usar
Conservador	0.15 – 0.20	Baixo	Alto	Alto	Prioriza segurança. Recusa mais clientes, mas
Moderado	0.25 – 0.35	Médio	Médio	Médio	Ponto de equilíbrio. Maximiza a separação entr
Agressivo	0.40 – 0.50	Alto	Baixo	Baixo	Prioriza volume. Aprova mais clientes, inclusive

Como calcular cada cenário em Python

```
import pandas as pd
import numpy as np

def simular_politica(y_true, y_prob, threshold, ticket_medio=5000):
    y_pred = (y_prob >= threshold).astype(int)

    aprovados_mask = y_pred == 0
    total = len(y_true)
    aprovados = aprovados_mask.sum()
    inadimplentes_aprovados = y_true[aprovados_mask].sum()

    taxa_inadimplencia = inadimplentes_aprovados / aprovados if aprovados > 0 else 0
    perda_estimada = inadimplentes_aprovados * ticket_medio
    volume_aprovado = aprovados / total

    return {
        'threshold': threshold,
```

```
'aprovados': aprovados,
'volume_aprovado_%': f'{volume_aprovado:.1%}',
'inadimplentes_aprovados': inadimplentes_aprovados,
'taxa_inadimplencia_%': f'{taxa_inadimplencia:.1%}',
'perda_estimada_R$': f'R$ {perda_estimada:,.0f}',
}

# Rodar os três cenários
resultados = []
for t in [0.15, 0.30, 0.45]:
    resultados.append(simular_politica(y_test, y_prob, t))

df_politica = pd.DataFrame(resultados)
print(df_politica.to_string(index=False))
```

8

Implementação Passo a Passo em Python

Código completo e comentado para o projeto Give Me Some Credit

Antes de começar

Execute cada bloco na ordem apresentada. Todos os gráficos são salvos automaticamente em `reports/figures/` — nenhuma janela interativa é necessária.

Passo 1 — Imports e configuração

```
import shap
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from pathlib import Path

# Garantir que a pasta de figuras existe
Path('reports/figures').mkdir(parents=True, exist_ok=True)

# Inicializar o JS do SHAP (necessário para alguns plots)
shap.initjs()

# Configuração de estilo dos gráficos
plt.rcParams['figure.dpi'] = 150
plt.rcParams['savefig.bbox'] = 'tight'
```

Passo 2 — Criar o explainer e calcular SHAP values

```
# lgbm_model = seu modelo LightGBM treinado
# X_test = features do conjunto de teste
# y_test = labels reais do conjunto de teste
# y_prob = probabilidades previstas pelo modelo

# Amostrar para eficiência (500-2000 obs são suficientes para visualização)
np.random.seed(42)
sample_idx = np.random.choice(len(X_test), size=1000, replace=False)
X_sample = X_test.iloc[sample_idx]
y_sample = y_test.iloc[sample_idx]
y_prob_sample = y_prob[sample_idx]

# Criar explainer e calcular SHAP values
explainer = shap.TreeExplainer(lgbm_model)
shap_values = explainer.shap_values(X_sample)

# shap_values é uma lista: [classe_0, classe_1]
# Sempre usar índice [1] para inadimplência
sv = shap_values[1] # shape: (1000, n_features)
```

```
# API moderna – necessária para waterfall
explanation = explainer(X_sample)
```

Passo 3 — Beeswarm (importância global)

```
plt.figure(figsize=(10, 6))
shap.summary_plot(
    sv,
    X_sample,
    plot_type='dot',
    max_display=10,
    show=False
)
plt.title('Importância Global das Features – SHAP Beeswarm', fontsize=13)
plt.savefig('reports/figures/shap_beeswarm.png')
plt.close()
print('Beeswarm salvo.')
```

Passo 4 — Waterfall para cliente de alto risco

```
# Encontrar cliente com maior score (mais risco)
idx_alto = y_prob_sample.argmax()

print(f'Cliente alto risco – Score: {y_prob_sample[idx_alto]:.3f}')
print(f'Label real: {y_sample.iloc[idx_alto]} (1=inadimplente)')
print()
print('Top 5 SHAP values:')
sv_cliente = sv[idx_alto]
feat_names = X_sample.columns.tolist()
shap_df = pd.DataFrame({'feature': feat_names, 'shap': sv_cliente})
shap_df = shap_df.reindex(shap_df['shap'].abs().sort_values(ascending=False).index)
print(shap_df.head())

# Plotar waterfall
plt.figure(figsize=(10, 6))
shap.plots.waterfall(explanation[idx_alto], show=False)
plt.title('Explicação Individual – Cliente de Alto Risco')
plt.savefig('reports/figures/shap_waterfall_alto_risco.png')
plt.close()
```

Passo 5 — Waterfall para cliente de baixo risco

```
# Encontrar cliente com menor score (menos risco)
idx_baixo = y_prob_sample.argmin()

print(f'Cliente baixo risco – Score: {y_prob_sample[idx_baixo]:.3f}')
```



```
print(f'Label real: {y_sample.iloc[idx_baixo]} (0=adimplente)')

plt.figure(figsize=(10, 6))

shap.plots.waterfall(explanation[idx_baixo], show=False)

plt.title('Explicação Individual – Cliente de Baixo Risco')

plt.savefig('reports/figures/shap_waterfall_baixo_risco.png')

plt.close()
```

Passo 6 — Extrair importâncias para o relatório

```
# Importância média absoluta de cada feature
mean_abs_shap = np.abs(sv).mean(axis=0)

importance_df = pd.DataFrame({
    'feature': X_sample.columns,
    'mean_abs_shap': mean_abs_shap
}).sort_values('mean_abs_shap', ascending=False)

print('Ranking de importância global:')
print(importance_df.to_string(index=False))

# Salvar como CSV para o relatório
importance_df.to_csv('reports/shap_importance.csv', index=False)
```

9

Erros Comuns e Boas Práticas

O que evitar e o que demonstra maturidade analítica

Erros que aparecem em entrevistas

- ✗ Usar `shap_values[0]` em vez de `shap_values[1]`**
Em problemas binários, `shap_values` é uma lista de dois arrays. O índice [0] corresponde à classe negativa (adimplente). Para análise de risco, sempre usar `shap_values[1]` (inadimplente).
- ✗ Calcular SHAP no test set inteiro sem amostrar**
Para bases grandes, isso pode levar horas. Uma amostra de 500 a 2.000 observações é suficiente para visualização e representa bem a distribuição global.
- ✗ Confundir importância com causalidade**
SHAP mede correlação/associação, não causa. Uma feature com SHAP alto influencia o modelo, mas não necessariamente causa a inadimplência na realidade.
- ✗ Parar nos gráficos sem narrativa**
O gráfico sozinho não convence ninguém. Cada visualização precisa de uma frase explicando o que ela significa para a política de crédito — esse é o diferencial.
- ✗ Não verificar se o ranking faz sentido de negócio**
Se uma feature irrelevante aparece no topo do beeswarm, pode ser sinal de data leakage ou overfitting. Sempre questionar: faz sentido essa feature ser tão importante?

Boas práticas que demonstram maturidade

- ✓ Verificar o valor esperado do explainer**
`explainer.expected_value` deve ser próximo da taxa de inadimplência da base (~0.067). Se estiver muito diferente, algo está errado no pipeline.
- ✓ Documentar o `expected_value` no relatório**
Mostrar que `previsão_cliente = expected_value + soma(shap_values)` demonstra que você entende a propriedade de eficiência do SHAP.
- ✓ Usar a mesma amostra para todos os plots**
Fixar `random_state=42` e usar sempre o mesmo `X_sample` garante que os gráficos são comparáveis entre si.
- ✓ Salvar os SHAP values calculados**
Calcular é lento. Salvar com `np.save('shap_values.npy', sv)` permite replotar sem recalculá-los.

Conectar SHAP com EDA



As features mais importantes no SHAP devem ser as mesmas que mostraram maior variação na análise bivariada. Inconsistências merecem investigação.

Resumo final

Você está pronto para implementar. O fluxo é: instalar shap → TreeExplainer → calcular shap_values → beeswarm global → waterfall individual → simular política de threshold. Cada etapa tem código pronto no Capítulo 8.